PHY2650 Homework1

Chen Yilun(123090048)

February 4, 2025

All codes please refer to the attached python files. The specific content of all code shall be subject to the Python file! The code in the PDF is for display and reference only.

1 Q1

HW1

Q_1 $x = \frac{a}{b}$ where $\tilde{a} = a(1+\delta a)$, $\tilde{b} = b(1+\delta a)$

$$\Rightarrow \delta x = \frac{\frac{\tilde{a}}{\tilde{b}} - \frac{a}{b}}{\left(\frac{a}{b}\right)} = \frac{\frac{a(1+\delta a)}{b(1+\delta b)} - \frac{a}{b}}{\frac{a}{b}} = \frac{1+\delta a}{1+\delta b} - 1$$

$$= \frac{1+\delta a-(1+\delta b)}{1+\delta b} = \frac{\delta a-\delta b}{1+\delta b} = (\delta a-\delta b)(1+\delta b)^{-1}$$

$$\approx (\delta a-\delta b)(1-\delta b) \approx \delta a-\delta b \quad (\text{omiting } \delta a\delta b, (\delta b)^2)$$

Hence, by the triangle inequality

$$\delta x = \delta a-\delta b \leq |\delta a| + |\delta b| = \varepsilon_a + \varepsilon_b$$

where $\varepsilon_a, \varepsilon_b$ are relative error bound

hence the error bound of x is $\varepsilon_x = \varepsilon_a + \varepsilon_b$

2 Q2

2.1 Code

```python
import numpy as np
import matplotlib.pyplot as plt  # Import the pyplot module
import math


def f(x):
    """
    Define the function f(x)
    :param x: The input independent variable
    :return: The result of the function calculation
    """
    return x**6 + 0.1 * np.log(np.abs(1 + 3 * (1 - x)))
```

```
numpoints = 100  # Define the number of points used to plot the function

x = np.linspace(0.5, 1.5, numpoints)  # Generate numpoints in the interval

y = f(x)

plt.xlabel("x")  # Set the label of the x-axis
plt.ylabel("f(x)")  # Set the label of the y-axis

plt.plot(x, y)  # Plot the function graph
plt.show()  # Display the function graph

k = 1.3  # Initialize the variable k to 1.3
while k <= 1.4:
    print(k, f(k))
    k += 0.01  # Increase the value of k by 0.01
```

2.2 Plot

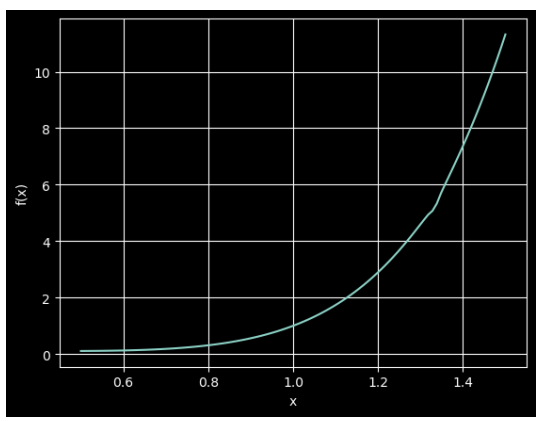

Figure 1: plot of f(x)

The "dip" occurs at $x = \frac{4}{3}$.
The "dip" of the function is mainly due to the logarithmic part.
When x approaches $\frac{4}{3}$,
the logarithmic function $y = 0.1\log(|4 - 3x|)$
decreases rapidly because the logarithmic function approaches negative infinity
as it approaches zero from the positive direction.
At this time,
the growth of the polynomial part (x^6)
is not fast enough to offset the decreasing trend of the logarithmic part.
When x exceeds $\frac{4}{3}$,
the growth of the polynomial part dominates,
and the function starts to rise again.

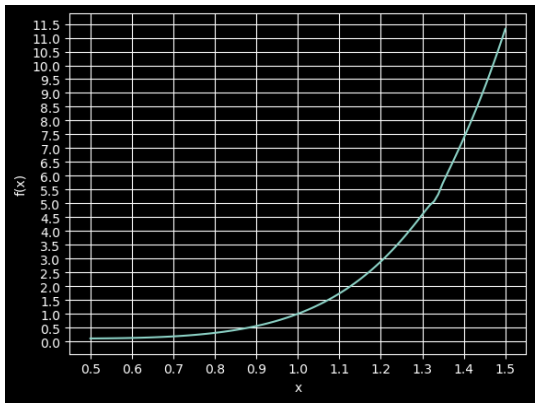

Figure 2: plot of f(x) with finer grid

3 Q3

```
def searchSquareNum(n):
    numList = []  # Initialize an empty list to store square numbers

    try:
        n = int(n)
    except:
        print('Please input only an integer')
        #send error message if the input n is not an integer

    k = 1
    m = 1

    while True:
        k = m ** 2  # Calculate the square of m
        if k >= n:  # Check if square is greater than or equal to n
            break
        numList.append(k)  # Append the square to numList
        m += 1  # Increment m

    print(f'The square numbers up to {n} are {numList}')
    # Print the result in format
    return numList


searchSquareNum(1000)
```

4 Q4

```
import math


def taylorSineTerm(x, N):
    # Calculate the Nth term of the Taylor series for sine
    return ((-1) ** N) * (x ** (2 * N + 1)) / math.factorial(2 * N + 1)
```

```
def taylorExpansionOfSin(x):
    sum = 0  # Initialize the sum of the Taylor series
    N = 0  # Initialize the term index
    if x == 0:
        print(0)
        print(f'the summation ends at N = {N}')
        return 0
    while abs(taylorSineTerm(x, N) * (10 ** 7)) >= abs(sum):
        # Add the Nth term to the sum if it's significant
        sum += taylorSineTerm(x, N)
        N += 1
        # print(taylorSineTerm(x, N))
    print(sum)
    print(f'the summation ends at N = {N}')
    return sum


def difference(x):
    # Calculate the difference between the Taylor expansion
    # and the actual value
    diff = abs(taylorExpansionOfSin(x) - math.sin(x))
    print(f'the difference is {diff}')
    return diff


# Try x = 0.5
taylorExpansionOfSin(0.5)
print()
difference(0.5)
```

5 Q5

Here are the values and error when x is from 1 to 100:

```
x = 1, N = 5, sum = 0.8414710097001764, error = 2.4892279903987458e-08
x = 2, N = 7, sum = 0.9092974515196738, error = 2.469399207338796e-08
x = 3, N = 9, sum = 0.1411200174132425, error = 9.353375296505817e-09
x = 4, N = 11, sum = -0.7568024926569087, error = 2.6510195150208915e-09
x = 5, N = 12, sum = -0.9589242932128198, error = 1.854968134651358e-08
x = 6, N = 14, sum = -0.27941550221018424, error = 4.011258381009242e-09
x = 7, N = 15, sum = 0.6569866170512744, error = 1.8332485307581692e-08
x = 8, N = 16, sum = 0.9893581773748686, error = 6.924851314593639e-08
x = 9, N = 18, sum = 0.4121184712778701, error = 1.3963886491552557e-08
x = 10, N = 19, sum = -0.5440210646963509, error = 4.619301885444571e-08
x = 10, N = 19, sum = -0.5440210646963509, error = 4.619301885444571e-08
x = 11, N = 21, sum = -0.9999901971573234, error = 9.393380073774438e-09
x = 12, N = 22, sum = -0.536572946655428, error = 2.8654993045051924e-08
x = 13, N = 24, sum = 0.4201670309237957, error = 5.902845212624186e-09
x = 14, N = 25, sum = 0.990607372754533, error = 1.7059662682505916e-08
x = 15, N = 26, sum = 0.6502877933350776, error = 4.6822039245775215e-08
x = 16, N = 28, sum = -0.28790332666974433, error = 1.0004679029673724e-08
x = 17, N = 29, sum = -0.9613974655323653, error = 2.6347191539599635e-08
```

```
x = 18, N = 30, sum = -0.7509873140223561, error = 6.725067991375511e-08
x = 19, N = 33, sum = 0.14987721106093257, error = 1.3979802315766676e-09
x = 20, N = 33, sum = 0.9129452856899325, error = 3.496230482902263e-08
x = 21, N = 35, sum = 0.8366556502462718, error = 1.1710215686022707e-08
x = 22, N = 38, sum = -0.00885130517331938, error = 4.117084496241263e-09
x = 23, N = 37, sum = -0.8462204239602107, error = 1.9785040050734892e-08
x = 24, N = 39, sum = -0.9055783071852938, error = 5.482133003109624e-08
x = 25, N = 41, sum = -0.1323523332756863, error = 5.831779132658976e-07
x = 26, N = 41, sum = 0.7625585677098587, error = 1.1723025605814286e-07
x = 27, N = 43, sum = 0.9563779283175255, error = 1.999913022521227e-06
x = 28, N = 45, sum = 0.2709060649291043, error = 2.766212352511843e-07
x = 29, N = 46, sum = -0.6636555417814031, error = 2.165756843552913e-05
x = 30, N = 47, sum = -0.9882783116573879, error = 0.0002466875645260602
x = 31, N = 49, sum = -0.4039467007790409, error = 9.094454402414076e-05
x = 32, N = 50, sum = 0.5520087616982938, error = 0.00058208045660324
x = 33, N = 51, sum = 0.9980331969001163, error = 0.0018786632071509102
x = 34, N = 53, sum = 0.5193320744349768, error = 0.009750611685047095
x = 35, N = 54, sum = -0.43441292980413043, error = 0.006230260307979407
x = 36, N = 55, sum = -0.9650706547132785, error = 0.026708198729837318
x = 37, N = 56, sum = -0.7565588938026231, error = 0.11302076044562359
x = 38, N = 58, sum = 0.3960663661733253, error = 0.09969778746393998
x = 39, N = 60, sum = 0.3236822919430559, error = 0.6401130943410318
x = 40, N = 60, sum = -3.2822092133102636, error = 4.027322373789612
x = 41, N = 61, sum = -2.812198282315549, error = 2.6535756135108404
x = 42, N = 62, sum = -11.189445057817396, error = 10.272923509901762
x = 43, N = 63, sum = -67.27444478273009, error = 66.44267004010149
x = 44, N = 64, sum = -69.08329863432868, error = 69.1010005594341
x = 45, N = 66, sum = 37.93452239839774, error = 37.08361887386362
x = 46, N = 66, sum = 256.61056770958965, error = 255.70877936194083
x = 47, N = 67, sum = -762.0169304979033, error = 762.1405036206486
x = 48, N = 67, sum = 3458.5643596590594, error = 3459.332614320383
x = 49, N = 68, sum = -19811.728971354867, error = 19810.775218702107
x = 50, N = 69, sum = -25906.59440991337, error = 25906.332035059666
x = 51, N = 70, sum = -172896.6928918742, error = 172897.36312105006
x = 52, N = 71, sum = 441930.06314069906, error = 441929.076513107
x = 53, N = 72, sum = 798689.1039335495, error = 798688.7080083993
x = 54, N = 74, sum = 201091.35277015856, error = 201091.9115592074
x = 55, N = 74, sum = 774825.1672000011, error = 774826.1669551744
x = 56, N = 74, sum = 19562797.674335428, error = 19562798.19588643
x = 57, N = 76, sum = -13733634.27434589, error = 13733634.710510645
x = 58, N = 76, sum = -46164506.101266704, error = 46164507.09413935
x = 59, N = 77, sum = -394867750.18983245, error = 394867750.82657045
x = 60, N = 77, sum = -1040182759.7300932, error = 1040182759.4252826
x = 61, N = 78, sum = -1902532877.5632563, error = 1902532876.5971384
x = 62, N = 79, sum = 13427775720.745657, error = 13427775721.484838
x = 63, N = 80, sum = 8053116623.358149, error = 8053116623.190793
x = 64, N = 81, sum = -63969027487.9773, error = 63969027488.89733
x = 65, N = 81, sum = -194932344570.60422, error = 194932344571.43106
x = 66, N = 83, sum = -249359088183.4633, error = 249359088183.43674
x = 67, N = 83, sum = 1470799996627.6624, error = 1470799996628.5178
x = 68, N = 84, sum = 1736627568267.4587, error = 1736627568268.3567
x = 69, N = 85, sum = -7179210615847.565, error = 7179210615847.45
x = 70, N = 87, sum = -3164386588246.2295, error = 3164386588247.0034
x = 71, N = 89, sum = -519990991575.5949, error = 519990991576.54596
```

```
x = 72, N = 88, sum = 18817914726329.14, error = 18817914726328.887
x = 73, N = 90, sum = 21223128584701.527, error = 21223128584702.203
x = 74, N = 89, sum = 389913248090217.0, error = 389913248090218.0
x = 75, N = 90, sum = 1162792272048649.2, error = 1162792272048649.8
x = 76, N = 90, sum = -8205109641752168.0, error = 8205109641752169.0
x = 77, N = 91, sum = 2.488303381721387e+16, error = 2.488303381721387e+16
x = 78, N = 92, sum = -2.905294170961408e+16, error = 2.905294170961408e+16
x = 79, N = 93, sum = 8.430036165125453e+16, error = 8.430036165125453e+16
x = 80, N = 94, sum = 9.487840541325083e+16, error = 9.487840541325083e+16
x = 81, N = 95, sum = 8.817233029809887e+17, error = 8.817233029809887e+17
x = 82, N = 96, sum = -1.1187322291849094e+18, error = 1.1187322291849094e+18
x = 83, N = 96, sum = 4.968939622441953e+18, error = 4.968939622441953e+18
x = 84, N = 97, sum = -2.0025445713603867e+19, error = 2.0025445713603867e+19
x = 85, N = 98, sum = -5.408322702569982e+19, error = 5.408322702569982e+19
x = 86, N = 98, sum = -1.4483709569991347e+20, error = 1.4483709569991347e+20
x = 87, N = 99, sum = 2.719619975211302e+20, error = 2.719619975211302e+20
x = 88, N = 100, sum = 6.83289687346446e+20, error = 6.83289687346446e+20
x = 89, N = 100, sum = -4.804950979144479e+21, error = 4.804950979144479e+21
x = 90, N = 102, sum = -7.005429166281726e+21, error = 7.005429166281726e+21
x = 91, N = 105, sum = 1.496404659458053e+20, error = 1.496404659458053e+20
x = 92, N = 103, sum = -5.850410323941645e+22, error = 5.850410323941645e+22
x = 93, N = 104, sum = 9.658404941266049e+22, error = 9.658404941266049e+22
x = 94, N = 106, sum = 4.138936437904746e+22, error = 4.138936437904746e+22
x = 95, N = 107, sum = -7.903539633668151e+22, error = 7.903539633668151e+22
x = 96, N = 107, sum = -1.5972826674735094e+24, error = 1.5972826674735094e+24
x = 97, N = 107, sum = 3.3126774191974173e+24, error = 3.3126774191974173e+24
x = 98, N = 107, sum = -3.335885370084312e+25, error = 3.335885370084312e+25
x = 99, N = 108, sum = -7.349906874266141e+25, error = 7.349906874266141e+25
x = 100, N = 110, sum = 3.6809885386809145e+25, error = 3.6809885386809145e+25
```

From above we can observe that
when x <= 24, the relative error is of 10^{-9} or 10^{-8};
when x>=28, the relative error started to get larger than 10^{-7};
when x>=32, the relative error is above 10−3, where the accuracy become worse;
when x>=39, the relative error is comparable or larger than 1 and the accuracy is totally lost.
When x>=40, the series no longer converges since the output of the sum is
larger than or much larger than 1.
If x continues to grow, the sum and error are ridiculously large.

6 Q6

6.1 Code

```
import math
import numpy as np
import matplotlib.pyplot as plt

# Function to calculate the n-th term of the Taylor series for sin(x)
def taylorSineTerm(x, n):
    return ((-1) ** n) * (x ** (2 * n + 1)) / math.factorial(2 * n + 1)

# Function to calculate the error of the Taylor
series approximation of sin(x) up to N terms
def taylorExpansionOfSin(x,N):
```

```
    sum_result = 0
    n = 0
    while n <= N:
        term = taylorSineTerm(x, n)
        sum_result += term
        n += 1
    error = abs(sum_result - math.sin(x))
    return error


# List of x values to calculate errors for
x_values = [0.5, 1, 2, 5, 10]

for x in x_values:
    N_values = []
    Error_values = []
    max_N = 20

    # Calculate errors for different numbers of terms (N)
    for N in range(1,max_N):
        error = taylorExpansionOfSin(x,N)
        N_values.append(N)
        Error_values.append(error)

    N_values = np.array(N_values)
    Error_values = np.array(Error_values)

    plt.plot(N_values, Error_values, label=f'x = {x}')

plt.xlabel('N')
plt.ylabel('Error')

plt.xticks(range(0, max_N + 1, 1))   # Set x-axis tick marks
plt.ylim(0, 0.3)

plt.title('Error vs N for different x values')  # Set y-axis limits
plt.legend()  # Show legend for x with different colors

plt.show()
```

6.2 Plot

7 Q7

7.1 Code

```
import math
import numpy as np
import matplotlib.pyplot as plt

# Initialize variables
D = {} #The dictionary stores N as keys and results as values
Errors = {}  #The dictionary stores N as keys and errors as values
```

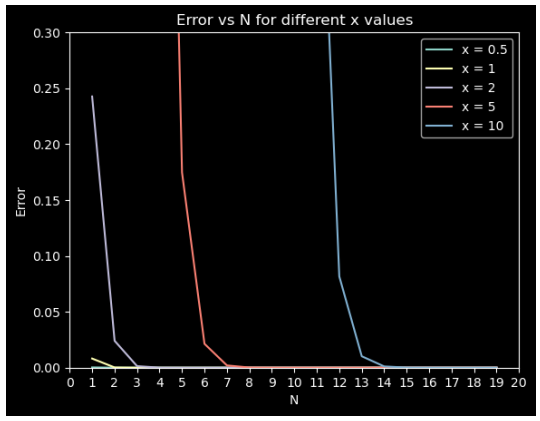

Figure 3: plot of error vs. N

```
# Define the function xloop to calculate x for different N values
def xloop():
    # Loop through N from 10 to 100
    for N in range(10, 101):
        # Reset x to 1/2 at the start of each N iteration
        x = 1/2
        # Perform N nested calculations
        for _ in range(1, N + 1):
            x = 1 / (2 + x)
        # print(f'When N = {N}, x = {x}')
        # Store the current N and x in dictionary D
        D[N] = 1 + x
        Errors[N] = abs((1+x) - math.sqrt(2))


xloop()



# Extract N values and corresponding results
N_values = list(D.keys())
result_values = list(D.values())
# Transform into arrays
N_values = np.array(N_values)
result_values = np.array(result_values)



# Plot the results as a function of N
plt.figure(figsize=(10, 6))
plt.plot(N_values, result_values)
plt.title('Results vs N')
plt.xlabel('N')
plt.ylabel('Result (x)')
plt.grid(True)
```

```
# Show the plot
plt.show()
```

```
# Extract N values and corresponding errors
N_values = list(Errors.keys())
error_values = list(Errors.values())
# Transform into arrays
N_values = np.array(N_values)
error_values = np.array(error_values)
```

```
# Plot the errors as a function of N
plt.figure(figsize=(10, 6))
plt.plot(N_values, error_values)
plt.title('Errors vs N')
plt.xlabel('N')
plt.ylabel('Error (x)')
plt.grid(True)
```

```
# Show the plot
plt.show()
```

7.2 Plot1

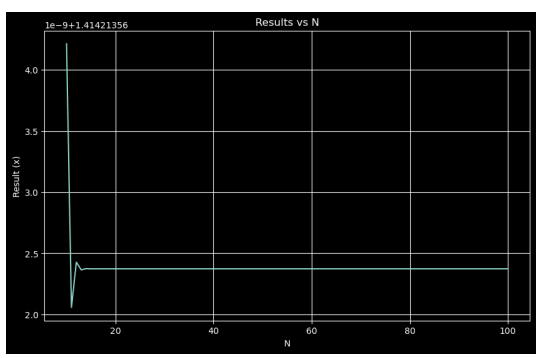

Figure 4: plot of result vs. N

7.3 Plot2

8 Q8

8.1 Code

```
import math

def singleContribution(i, j, k):
```

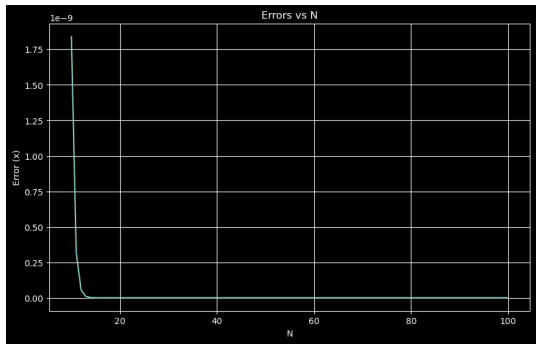

Figure 5: plot of error vs. N

```
    '''Define a function to calculate the contribution of a single atom
    to the Madelung constant'''
    if i == 0 and j == 0 and k == 0:
        return 0

    absContribution = math.sqrt(i**2 + j**2 + k**2)
    #calculate the absolute value of the contribution

    if (i + j + k) % 2 == 0: #if the sum of i, j, k
    is even,
    the contribution is positive
        return 1 / absContribution
    else:                    #if the sum of i, j, k is odd,
    the contribution is negative
        return -1 / absContribution


def getMadelung(L):
    ''''Define a function to calculate the Madelung constant
    for a range of L'''
    sum = 0
    for i in range(-L, L+1):
        for j in range(-L, L+1):
            for k in range(-L, L+1):
                sum += singleContribution(i, j, k)
                #sum the contribution of each atom
    return sum

L = 200 #set the range of L
print(getMadelung(L))
#the result is -1.7446850421707383 when L = 200
#using C++ can get the result faster than python
```

8.2 Result

The result is -1.7446850421707383 or the absolute value 1.7446850421707383